

osu!mania 4K 难度分级系统

工作纪实与分级再设计研究

生成：Claude Code（本地）

2026-06-26

阅读指南 本报告分两部分。**第一部分**是从沙箱迁到本地以来做的全部工作的完整纪实（环境、引擎、自动化、开源、yumu 研究、维度工程、关键验证）。**第二部分**是按你要求做的分级系统再设计研究：如果不一一定要锚定、如果从零重做、如果不把 dan 当绝对标准，可以怎么设计。附录是数据速览、文件与命令清单、术语表。

Contents

1 引言：从沙箱到本地	3
2 本地化工程	3
2.1 环境搭建	3
2.2 踩到的关键坑：MSIX 文件系统重定向	3
2.3 引擎本地化改造	3
2.4 计算器零漂移验证	4
3 运行与交付	4
3.1 realm 解析与增量	4
3.2 collection.db（含手工夹保留）	4
3.3 中文 PDF 报告	4
3.4 交付前自检（§ 16）	4
4 自动化与易用	4
4.1 每晚计划任务	5
4.2 一键启动器 osu4k.cmd（与 cmd 中文坑）	5
5 开源发布	5
6 yumu-bot 研究	5
6.1 它的多套体系	5
6.2 它参考了什么（你问的）	5
6.3 可借鉴清单（精选）	5
6.4 明确不适用	6
7 维度工程：MinaCalc 8 skillset	6
7.1 免费维度的发现	6
7.2 回填全库（零漂移）	6
7.3 数据分析	6

8 关键验证：REFORM 阶梯无法区分公式	8
第二部分：分级系统再设计研究	9
9 现行公式与它的四个隐含假设	9
10 关于“锚定”：要不要、怎么锚	9
11 不锚定的做法：分布 / 分位归一	9
12 不把 dan 当绝对标准	10
13 用你自己的成绩校准难度	10
14 从零重做：四套蓝图	11
15 推荐与权衡	11
A 附录 A：数据速览	13
B 附录 B：文件与命令清单	13
C 附录 C：术语表	13

1 引言：从沙箱到本地

这套工具的目标：把 osu!lazer 里所有 4K（4 键）谱面按一套**复合难度**重新定级，分到 0.5 宽的难度桶（4k-00.0 ...4k-19.0），产出**可导入 lazer 的 collection.db** 和一份**中文 PDF 报告**；支持增量（只算新谱），可每晚自动运行。

复合难度的核心公式（用户既定）：

$$\text{base} = \frac{\text{MSD} + 4 \cdot \text{ISR} + 4 \cdot \text{RSR}}{9}, \quad \text{scaled} = 1.30 \cdot \text{base} \cdot (1 + 0.18 \ln r + 0.12 \text{hb})$$

其中 $\ln r$ = 长条占比， $\text{hb} = 4 \ln r(1 - \ln r)$ ，桶 = $\lfloor \text{scaled} \cdot 2 \rfloor / 2$ 。三个难度来源：**MSD**（Etterna MinaCalc）、**ISR**（Interlude/YAVSRG）、**RSR**（sunny rework）。锚点：REFORM 10 段 ≈ 13 。

此前这套流程跑在一个沙箱里（通过 Dropbox 镜像读 osu 文件夹），踩到的坑几乎都来自环境而非算法：同步延迟导致 realm 半截、单次命令 45 秒上限、环境易失。**本地直接读硬盘上的真实 client.realm，没有这些问题**——前提是读取时 lazer 已关闭。本次工作就是把它完整迁到本地 Windows，并在此基础上做研究与改进。

2 本地化工程

2.1 环境搭建

一次性装好并固定到真实（非 AppData 重定向）位置：

- Python 3.12（标准库即可；报告另需 matplotlib / fonttools / numpy）。
- Node.js v24：跑三个难度计算器（MinaCalc 是 WASM）。
- MiKTeX + XeLaTeX：编译中文 PDF（开启自动补包）。
- 中文字体：直接用系统自带 SimSun（正文）与 DengXian（标题/图表），不需要 Noto CJK。
- 计算器环境：从 calc_env.tar.gz 解包到 %USERPROFILE%\osu4k_calc（真实目录，计划任务可见）。

2.2 踩到的关键坑：MSIX 文件系统重定向

Claude Code 在本机是 **MSIX 打包应用**。诊断发现文件系统访问**按工具区分**：

- PowerShell 工具 = 真实文件系统。winget/pip 安装、写文件都落到真实 C:\Users\...\AppData\Local，计划任务等包外进程可见。
- Bash 工具 = MSIX 虚拟化视图。对 %LOCALAPPDATA% 的写入被重定向进包私有目录 ...\Packages\Claude_...\LocalCache\Local\...，包外完全看不到。

后果与对策：凡是要给计划任务用、要持久的东西，一律用 PowerShell 工具、放在真实非-AppData 目录。计算器最初用 Bash 的 tar 解包落进了虚拟层，发现后用 PowerShell 重新解包到 %USERPROFILE%\osu4k_calc。

2.3 引擎本地化改造

把原沙箱代码改成**跨平台、不破坏原逻辑**（用环境变量 + tempfile，posix 默认值保留）：

- run_4k.py: /tmp/* 临时文件改用系统 temp；计算器路径走 CALC_REPO 环境变量；blob 索引每次从 realm 重建（持久机器上避免陈旧索引漏掉新谱）。

- `make_report.py`: 编译目录改用系统 `temp`; 字体从 Linux Noto 路径改成 Windows 系统字体; 修了两个 Windows 专有 bug——`subprocess` 用 GBK 解码 `xelatex` 输出 (强制 UTF-8)、`matplotlib` 缺 U+2212 减号 (字体回退链 [DengXian, DejaVu Sans])。
- **字体真相**: `simsunb.ttf` 不是宋体粗体, 而是 SimSun-ExtB 生僻字字体 (缺常用字)。正文粗体改用 `fontspec` 的 `AutoFakeBold` 合成; 无衬线用真粗体 `Dengb.ttf`。
- 新增 PowerShell 编排 `run_workflow.ps1` (自定位、设 PATH/环境变量、realm 健康检查、osu 进程检查、跑两个 Python、拷贝交付、写日志)。

2.4 计算器零漂移验证

对 12 张已在缓存的谱用 `node` 重算 MSD/ISR/RSR, 与缓存值对比: **最大偏差** 8.9×10^{-16} (浮点精度极限)。即 `node v24` 在 Windows 下与原沙箱**逐位一致**, 分级口径零漂移。

3 运行与交付

3.1 realm 解析与增量

手写的二进制 Realm 解析器在本地完美工作: 解析出 **4360 个谱面** (全部 md5↔sha256 配对)、644 条成绩、40 个收藏夹, 耗时 0.03 秒。增量逻辑: 遍历当前曲库每个谱面 md5, 已在缓存 (ok/skip) 的跳过, 只对新 md5 计算。

3.2 collection.db (含手工夹保留)

按 `osu!stable` 二进制格式写出, 每个非空难度桶 = 一个收藏夹。**重要修正**: `spec` 里硬编码要保留的手工收藏夹 `Y.S.Z.D.` 在当前 `realm` 已不存在, 你换成了 `10SKIP`。于是把逻辑从“只认一个固定名字”改为**保留所有非 4k- 开头的手工收藏夹**——这样以后任何手工夹都不会在导入时被覆盖丢失。最终 db: 40 个收藏夹 (39 难度桶 + `10SKIP`), 2404 个成员, 0 尾随字节。

3.3 中文 PDF 报告

`matplotlib` 出图 + XeLaTeX 编译, 5 节内容: 难度分布堆叠柱 (叠加平均星/平均准确率双折线)、全库类型构成、逐桶明细表、成绩误差 vs 复合难度对数反转散点 (带 $\log_2$ 拟合, 实测“难度每 +1.57 误差翻倍”)、准确率随游玩推进 6 段折线、各类型平均准确率。全程中文零豆腐块。

3.4 交付前自检 (§ 16)

检查项	结果
scaled 公式: REFORM 10 锚定	13.01 - 13.12 $\approx$ 13 ✓
计算器零漂移 (vs 缓存)	8.9×10^{-16} ✓
collection.db 往返解析	40 夹 / 2404 成员 / 0 尾随 / <code>10SKIP</code> 保留 ✓
4K 数 = 桶内总数、无未分桶	2400 = 2400, 未分桶 0 ✓
client.realm 只读	大小 7602176、mtime 全程未变 ✓
PDF: 5 节齐全、中文无缺字	0 处 Missing character ✓

4 自动化与易用

4.1 每晚计划任务

Windows 计划任务 `osu4k-nightly-classify`，每天 23:30 增量运行 `run_workflow.ps1-ResetBaseline`。`osu!lazer` 在运行或 `realm` 半截/锁定时**优雅跳过**（退出码 7 = `REALM_BUSY`，不改任何文件）。已实际触发验证：`LastTaskResult=0`，包外 5.1 上下文跑通。日志写到 `%LOCALAPPDATA%\osu4k\logs`（非 Dropbox，避免同步锁文件）。

4.2 一键启动器 `osu4k.cmd`（与 `cmd` 中文坑）

为方便“双击即跑”，做了 `osu4k.cmd`：双击自动跑增量、显示 DONE/SKIP/FAIL、跑完暂停；终端调用则不暂停并返回标准退出码。**踩坑**：`cmd.exe` 解析含中文的 `.cmd` 会因 UTF-8 多字节把行解析搞乱，导致内部 `powershell -File` 那行被拆断、根本没执行却返回 0。对策：把 `.cmd` 写成**纯 ASCII 瘦启动器**，真正的中文进度由它调用的 PowerShell/Python 输出。（另一面：`.ps1` 含中文反而必须 UTF-8 带 BOM，否则 Windows PowerShell 5.1 解析失败——两者要求相反。）

5 开源发布

仓库发布到 <https://github.com/C1MON/OM-BucketSynthesisTier>（公开，分支 `main`）。发布前做了隐私扫描（无用户名/邮箱/token 泄漏），提交用 GitHub noreply 邮箱（不暴露真实邮箱），`.gitignore` 排除了 20MB 第三方计算器包、`__pycache__`、缓存备份与日志。后续维度工程的改动也已提交并推送（commit `ae692e6`）。

6 yumu-bot 研究

对中国 `osu` 社区主流 bot **yumu-bot** 做了源码级研究（7 个 agent 并行精读 `yumu-bot` / `yumu-image` / `yumu-docs`，关键结论回源核验）。

6.1 它的多套体系

`yumu` 有 PP-（PPMinus 多维技能分）、PP+、Skill、三套对战 Rating（斗力），以及一套自研的 **osu!mania 难度模型 SkillMania6**——后者与本工具思路高度同族：把谱拆成多维技能 → 各维非线性拟合 → 降序加权聚合成单一 rating → 落入“段位桶”。

6.2 它参考了什么（你问的）

核心算法是作者自研的启发式工程模型，不引任何论文。只有两处外部来源：

- **curve.fit**（在线最小二乘拟合站）：13 种 pattern 的幂律系数 $[a, b]$ 各附了一条拟合链接。方法可借，系数不能抄。
- **段位边界完全照搬社区 dan 课题包**：4K 米段 = DDMythical 的 Reform Dan、4K 面段 = Underjoy LN Dan、7K = 锦锦的 Regular/LN Dan。`yumu` 只是把这些课题包的难度梯度手工标成阈值。

这点很重要：连最成熟的社区 bot，其“难度真值”也只是把人工 dan 课题包手工标定一下——dan 在它那里也不是什么严格的连续真值。

6.3 可借鉴清单（精选）

定级：稳健聚合（取次高维加权、防单一极端维度刷分，源码核验 0.6·第2大+0.4·第3大+0.2·第4大）；落桶用 boundary 数组 + 桶内线性内插 + “+”后缀；acc 惩罚函数把低准确率成绩降权。**报**

告：维度固定配色（社区已认可的 RC/ST/SP/LN/CO/PR 色板，hex 已核验）；自动整数网格刻度；误差散点按谱型上色；进步曲线叠难度基线；逐桶三点摘要（max/中位/min）；每图配白话解读 + 局限性声明。

6.4 明确不适用

PP/PP+/全球排名/邻域人群百分位/对战斗力——都强依赖在线提交、他人数据或对战结构，与本地单玩家 4K 工具不兼容。其方法学（多指标→单标量→固定阈值分桶+命名配色、相对分位抗漂移、置信降权）已被提炼进可执行条目。

7 维度工程：MinaCalc 8 skillset

7.1 免费维度的发现

你的 MSD 计算器（MinaCalc）每张谱本来就一次算出 8 个技能分：Overall + Stream / Jumpstream / Handstream / Stamina / JackSpeed / Chordjack / Technical。但 batch_4k.mjs 只取了 Overall，另外 7 个直接扔掉了。这是离线、零额外计算成本、已经算好的现成维度体系。改 runner 一行即可启用。

7.2 回填全库（零漂移）

改 runner 返回完整 8 维对象，再用 msd-only 脚本对全库 2390 张谱回填那 7 个维度，16.4 秒跑完。合并时校验 Overall 与缓存 msd：漂移精确为 0（MinaCalc 完全一致）。缓存已备份。

7.3 数据分析

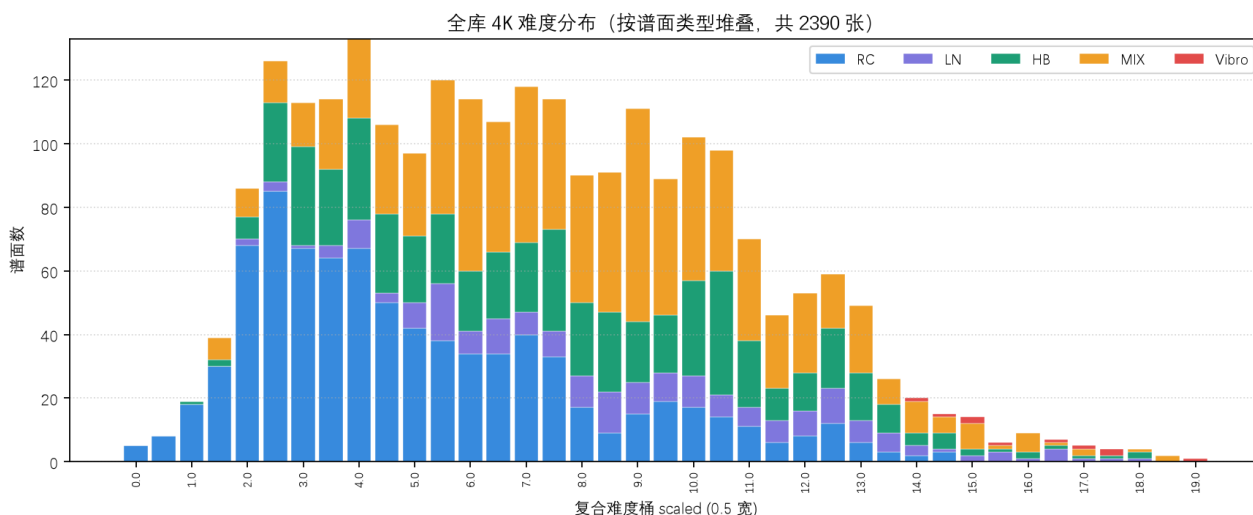


Figure 1: 全库 4K 难度分布（按谱面类型堆叠）。低-中难区谱面密集，高难区稀疏。

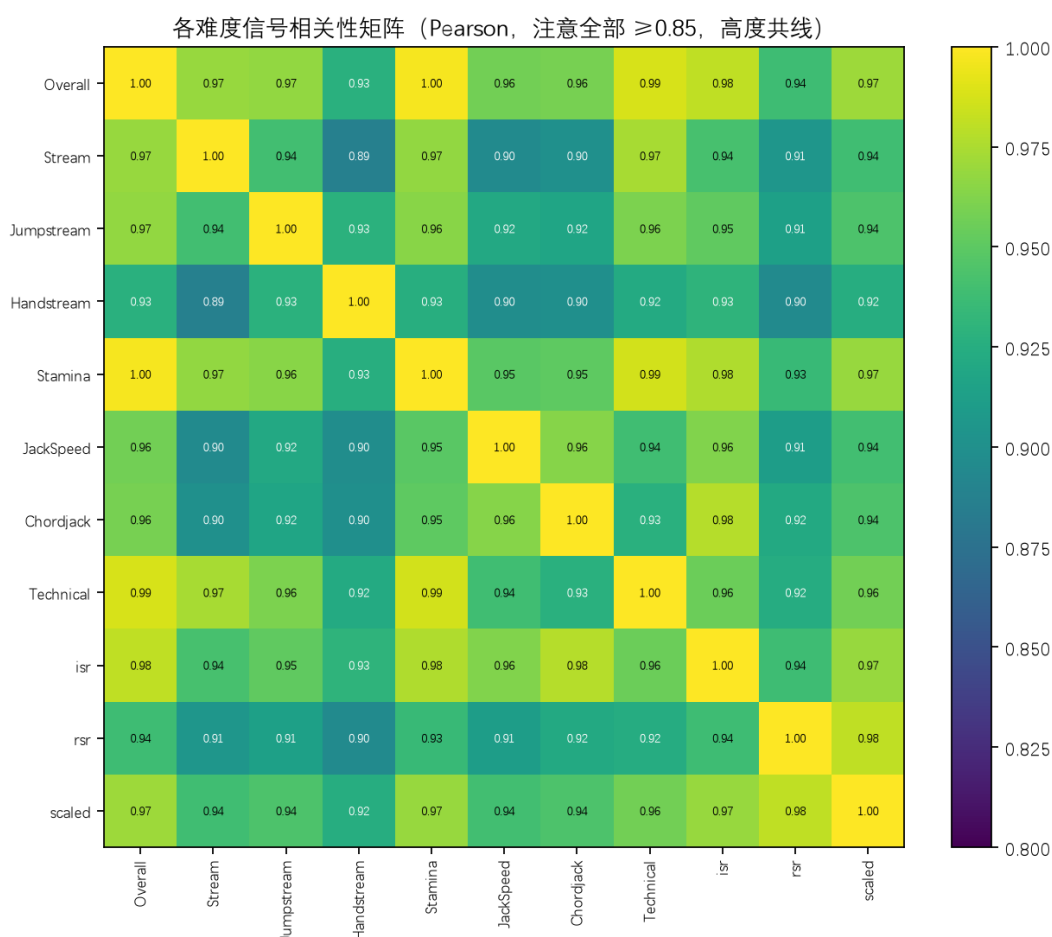


Figure 2: 各难度信号相关性矩阵。所有信号互相 ≥ 0.85 、与 scaled 多在 $0.94 - 0.98$ ——高度共线，这是后面关键验证的根源。

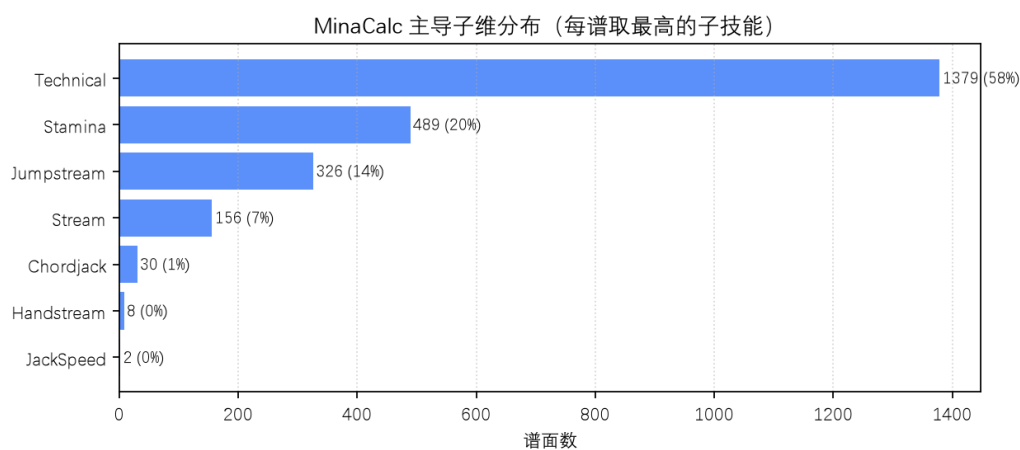


Figure 3: MinaCalc 主导子维分布: Technical 58% / Stamina 20% / Jumpstream 14% / Stream 7%, 纯叠键 (Chordjack/JackSpeed) 仅约 1%。

要点: MSD 中位 15.2、4-ISR 中位 15.1、4-RSR 中位 15.0——三者贡献完美平衡 (当初 1:4:4 就是为此)。且 Overall 中位等于 7 个子维的最大值 (差 0.00)——即 MSD 本来就 = 最难的那个技能, jack/chordjack 在 MSD 里没被低估。唯一看得见的偏差: 纯叠键谱 (如 Gulyayu, Overall 21.6 但 scaled 仅 9.8) 被 ISR/RSR 压低了。

8 关键验证：REFORM 阶梯无法区分公式

为判断“给维度更大权重是否更准”，用库里整套 REFORM dan 阶梯（1st...10th，社区公认的 4K 难度标尺）当真值检验。把每个候选公式都校准到 10th = 13.12，看阶梯是否单调、间距是否均匀。

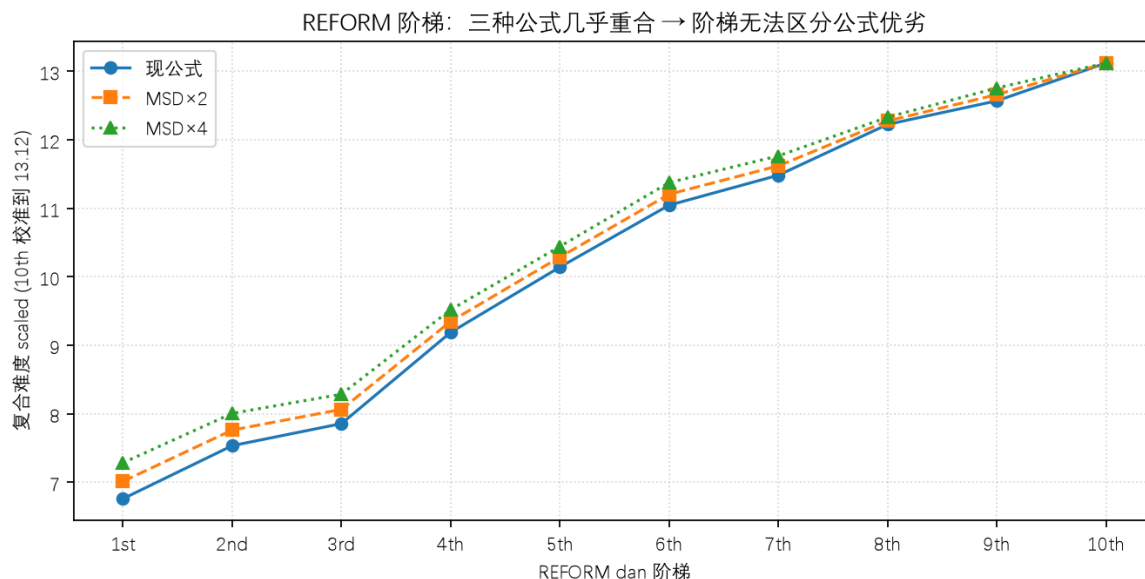


Figure 4: REFORM 阶梯在现公式 / MSD×2 / MSD×4 下几乎重合。三者逆序数都为 0、步长标准差都是 0.31。

结论（重要）：拿我们唯一的真值（REFORM 阶梯）检验，**重配权重并不更准**——各方案对人工阶梯的拟合度完全一致。原因：三个计算器 97% 共线，且 REFORM 阶梯是“全科同步”的（高段在各技能上都更难），所以无论怎么加权，阶梯形状都一样。**重配权重只对“偏科谱”有影响，而 REFORM 阶梯里几乎没有偏科谱，因此这把尺子分辨不出哪个公式更好。**这直接引出了第二部分的核心问题：既然 dan 阶梯分辨力这么弱，分级到底该靠什么定标？

附：MinaCalc 原理一句话。 它把“难度”定义为一个玩家需要多高实力，才能在这张谱上刚好打出目标准确率（默认 93%）——难度与玩家实力同尺。它逐行扫音符、按手型分 8 类技能建模、用“给定实力算预期准确率再反解所需实力”得分，并建模耐力衰减；参数由 Etterna 几百万真实成绩回归校准，故对 jack 评得准（osu 官方 strain 模型恰恰低估 jack）。

第二部分：分级系统再设计研究

9 现行公式与它的四个隐含假设

现行定级： $\text{base} = (\text{MSD} + 4 \text{ISR} + 4 \text{RSR})/9$, $\text{scaled} = 1.30 \cdot \text{base} \cdot (1 + 0.18 \ln r + 0.12 \text{hb})$, 落 0.5 桶。
它隐含了四个未必成立的假设：

1. **固定 1:4:4 权重**是对的——但第一部分已证明，三计算器 97% 共线，这个权重在 REFORM 阶梯上无法被验证为更优或更劣。
2. **单一标量足够**——把难度压成一个数，丢掉了 MinaCalc 已经免费算好的 8 维结构。
3. **REFORM 10 = 13 这个单点锚是可靠的**——把整条刻度的零点和单位全压在一个 dan 段上。
4. **dan 是连续难度真值**——而它其实是离散、不等距、随版本漂移的序数门槛。

你的三个问题正好对应假设 3、4 与“是否重做”。下面逐一展开。

10 关于“锚定”：要不要、怎么锚

先分清你要的是哪种刻度。绝对可比刻度(数字本身有意义、能跨谱比较、能对接官方星)需要锚定；相对排序刻度(只要“谁比谁难”)根本不需要锚。

一个数学事实(关键)。用单玩家成绩反推难度时，只有“你的能力 θ 减谱面难度 b ”这个差值可识别：把所有 θ 和 b 同时加一个常数 c ，预测完全不变。所以单玩家场景下“谱面绝对难度”在数学上不可识别，只能恢复到差一个全局平移+缩放。要得到对人有意义的绝对数字，必须用外部信息(MSD 量纲、或几张已知锚谱)把这个仿射变换钉死。

REFORM 10 = 13 单点锚的三个脆弱点：

- 该 dan 段本身是 4 首混合技能、pass/fail、随版本换曲的区间化噪声标签，它的误差会整体平移你的刻度。
- 第一部分已验证：REFORM 阶梯对所有候选公式单调且均匀度相同——它给不了“单位长度”的真实信息，用单点定一个单位等于拍脑袋。
- MinaCalc 一换版本，scaled 与 dan 的关系就漂移，单点锚无法自我诊断。

更稳的替代(按稳健度递增)：

1. **多锚最小二乘**：用 REFORM 全 10 段 + 官方星做锚集，拟合 $\text{target} \approx a + b \cdot x$ ；残差当每个锚的“可疑度”诊断，单个锚标锚被平均稀释。
2. **仿射对齐到 MSD 连续尺度**：把内部刻度线性回归到 MinaCalc Overall——锚“一整把尺子”而非“一个点”。
3. **固定参照 CDF 的分位归一**：根本不锚(见下一节)。

结论：放弃 REFORM 10 单点，改用“多锚回归到星/MSD + 分位刻度做展示备份”。

11 不锚定的做法：分布 / 分位归一

完全不依赖任何外部锚：难度 = 该谱在你整个谱库难度分布中的位置。

- **秩次 / 百分位**: 全库按原始难度排序, 谱 i 的刻度 $= \text{rank}_i / n$ 。只要新旧公式排序一致, 换公式 / 换 MinaCalc 版本, 玩家看到的刻度纹丝不动——对漂移几乎免疫。
- **正态分位变换**: $p_i = (\text{rank}_i - 0.5) / n$, 刻度 $= \Phi^{-1}(p_i)$, 把任意分布拉成正态, 间距更均匀。
- **分位数归一化**: 对 8 维 + ISR + RSR 各自按秩对齐到同一分布, 省去量纲对齐。
- **固定参照 CDF**: 保存某次全库分布作参照, 新谱按断点插值, 不重排老谱。

优点: 实现极简、全库覆盖无冷启动、对换版本免疫、不需要任何成绩。**缺点**: 刻度是“相对你的库”, 绝对/跨库语义弱(库整体偏难则数字被压); 不反映个人能力, 只反映相对排序。**这是现有数据下唯一“立刻可全覆盖、零外推”的方案。**

12 不把 dan 当绝对标准

dan 不宜当连续真值, 因为它在三个维度上都是带噪的离散序数门槛而非测量:

- **离散且不等距**: 一段 dan 是单血条连关约 4 首的 pass/fail, 相邻段难度差不均匀, 一段内 4 首彼此也不同 $\Rightarrow$ 单段对应一个难度区间而非一个点。
- **时间漂移**: 选曲随版本更换, 同名段位跨版本不可直接比。
- **混合性**: 一段混多种技能, “过了这段”不等于“这段恰好等于某难度值”。

你自己的验证就是直接证据: REFORM 阶梯对所有公式单调且均匀度相同, 即它**分辨力很弱**——能告诉你“单调对不对”, 几乎不能告诉你“单位多长 / 哪个公式更准”。

旁证: 连最成熟的体系也不把人工等级当真值。Eterna MinaCalc 的 321 维参数是开发者交互式手调(看哪个 skillset 高估了就 nerf), 官方明说“只是一般性指引, 不是绝对度量”; BMS 难度表靠社区两周一次投票形成序数共识; StepMania 难度的学术研究(Franks 等)直接把难度当序关系建模, 并发现“除 ITG 外所有方法都改进了原作者的人工标注”——人工标签普遍有噪、跨包不可比。

降格使用的具体做法:

1. **只当单调性约束**: 用 isotonic / 单调回归保证“更高 dan $\Rightarrow$ 预测难度不更低”, 而不把 dan 数值塞进最小二乘当等号目标。
2. **当软下界 / 低权锚**: 通过某 dan $\Rightarrow$ 对应 MSD 区间下限附近有通关力, 作为先验或多锚回归里的一个高方差(低权重)锚, 与官方星、成绩并列, 逆方差加权自动把它降权。
3. **显式记版本号**, 避免跨版本当同一真值。
4. **把你自己的成绩当主信号、dan 当稀疏校验锚**。

13 用你自己的成绩校准难度

这是“不靠 dan 也能定标”的核心: 把 Eterna MSD 的定义私有化到你一个人身上——难度 = “把你的准确率顶到固定参考点(如 93%)所需的能力刻度”。每张谱有自己的“acc $\leftrightarrow$ 难度”曲线, 难度就是这条曲线在参考 acc 处的反解值。

方法其实同源。它们都假设“成功概率是 $(\theta - b)$ 的单调函数”:

- **IRT / Rasch**: $P(\text{成功} | \theta, b) = \sigma(\theta - b)$, 给出生成模型与联合估计。
- **Elo / Glicko**: 就是 Rasch 的在线增量版——把每次游玩当一场“对局”, pass = 你赢。谱面用很小的 K (难度近似不变)、你自己用正常 K , 按日期顺序喂入即可。
- **逻辑 / 分位回归(explanatory IRT / LLTM)**: 把难度显式写成谱面特征的函数 $b = f(\text{MinaCalc 8 维}, \text{ISR}, \text{RSR}, \text{星没玩过的谱也能预测(冷启动)}, \text{分位回归还能给“难度置信带”})$ 。

单玩家的识别性陷阱(必须正视)。 只有一个玩家时, θ 对所有谱是同一个常数, 与所有 b 完全共线 $\Rightarrow$ 绝对难度不可识别。两条出路: (a) 外部锚(对齐 MSD / 选几张锚谱); (b) **时间漂移**——把你不同时期的成绩当成“不同的 θ ”(按月/季度分段), 一旦能力随时间变化, θ 与 b 不再完全共线, b 的相对结构就能被“撬”出来。

推荐架构(Duolingo AutoIRT 风格):两阶段。先用谱面特征(你已有的 8+2+1 维)训一个回归得每张谱的“特征预测难度”做先验(解决冷启动 + 稀疏兜底);再用你的 acc / 通关记录把打过的谱从先验往后验拉(样本多的拉得多, 没玩过的就用先验);每张谱输出**难度** $\pm$ **置信区间**;最后仿射锚到 MSD 量纲。

两个现实陷阱(决定它现在还不能当主刻度):

- **样本太薄:**约 535 - 644 条有效成绩 / 2400 张谱 $\Rightarrow$ 多数谱 0 - 2 条, 只有热门谱有 ≥ 5 条。时间分段只能切 2 - 4 段, 否则每段样本枯竭。
- **MNAR(非随机缺失):**当前只保留了 acc ≥ 0.80 的成功样本——“太难没碰”和“太易不刷”的谱系统性缺失, 会让模型**系统性低估难谱**。

要让它变可用, 得先改数据采集:放宽 acc 下限、保留低分与 fail/弃坑记录(才有“难度上界”信息)、持续累积数月。

14 从零重做：四套蓝图

蓝图	理念	锚定 / dan 角色	用你的成绩	工作量
A 成绩校准型 (个性化)	把 MSD 定义私有化: 难度=你达到参考 acc 所需能力	不靠单点; 对齐 MSD; dan 仅弱先验/校验	主用 (IRT/Elo)	高
B 分位归一型	难度=在你全库分布里的百分位; 完全不锚	显式不锚; dan 不用 (仅旁注)	可选/弱	低
C 多源融合型	8 维+ISR+RSR+星+dan+成绩 当多个带噪测量, 加权融合+置信区间	多锚回归; dan 作低权锚之一	作为一路源	中
D 最小改动型	保留现公式, 只在后面挂一层校准	多锚替单点; dan 作单调约束	轻量校验	低

A 成绩校准型:输出真正“对你而言”的难度, 能暴露 MinaCalc 与你体感分歧的谱(本工具独特价值)。但受单玩家识别性 + 样本薄 + MNAR 限制, 现阶段当不了主刻度。

B 分位归一型:最稳、最简、零外推、全库覆盖, 对换版本免疫。代价是“相对你的库”, 绝对语义弱。适合做面向玩家展示的主刻度。

C 多源融合型:用满所有信号, 输出置信区间(可标“此谱难度争议大”), 一个权重旋钮既能出“个人向”也能出“大众向”。注意三源 97% 共线, 融合必须正则化(岭回归/PCA), 否则逆方差会低估方差。

D 最小改动型:不重做, 只把脆弱的 REFORM 10 单点锚换成“REFORM 全阶梯 + 官方星的多锚 WLS”, 再用 isotonic 把原始 scaled 保单调映射到目标刻度。2400+ 谱样本足够, 今天就能上线, 换版本只重训校准层。

15 推荐与权衡

不要一步到位, 走渐进路线 B/D $\rightarrow$ C $\rightarrow$ A:

- **立刻(现有数据完全够、零外推):**上 **方案 D** (去掉 REFORM 10 单点锚, 改 REFORM 全阶梯 + 官方星多锚回归 + 一层 isotonic 校准) + 并行 **方案 B** (固定参照 CDF 的分位刻度, 作换版本免疫的展示主刻度)。这两者今天就能落地, **直接消除你最大的脆弱点——单点锚**。

- **中期(数据够):**升级到 **方案 C** 多源融合,岭回归把 8 维 + ISR + RSR + 星 + dan 融成带置信区间的刻度,dan 自动降为低权锚,并输出“置信区间最大 = 难度争议大”的谱列表。
- **终极(需先改数据采集):****方案 A** 个性化成绩校准是终极形态,但当前成绩偏薄且 MNAR;要让它稳,先放宽 acc 下限、记录 fail/弃坑、累积数月,再按时间分段撬动识别性。

一句话:现公式若不重做,最值得加的单项就是——用 REFORM 全阶梯多锚回归替掉“10 = 13”单点锚,再挂一层 isotonic 校准。这是用现有数据、最低成本、立刻把“脆弱定标”变稳的改动。而“维度”的最佳位置,短期是进报告(技能画像)与方案 C 的一路源,而非去重配那个被验证为“改了也不更准”的单一公式权重。

A 附录 A：数据速览

量	最小	中位	最大	均值
Overall (MSD)	0.00	15.18	37.90	15.19
Stamina	0.00	14.61	37.32	14.52
Technical	0.00	14.99	34.74	14.75
Jumpstream	0.00	13.72	33.12	13.61
Stream	0.00	12.82	32.75	12.98
Chordjack	0.00	9.54	37.16	10.12
Handstream	0.00	9.06	34.10	10.22
JackSpeed	0.00	7.94	26.82	8.27
ISR	0.00	3.78	15.20	4.10
RSR	0.00	3.76	9.53	3.80
scaled	0.00	7.04	19.24	7.32

与 scaled 的相关性：RSR +0.981、ISR +0.969、Overall +0.971、Stamina +0.969、Technical +0.958（其余子维 0.92 - 0.95）。样本：2390 张已回填维度的 4K 谱。

B 附录 B：文件与命令清单

交付目录 12_osu\4k_classification\：4k_collections.db、4k_report.pdf、osu4k.cmd（双击启动器）、README.md、4K_WORKFLOW_SPEC.md。

引擎 engine：run_workflow.ps1、run_4k.py、realm_engine.py、osu_meta.py、make_report.py、node_runner\batch_4k.mjs、cache\results.json。

计算器（真实目录）：%USERPROFILE%\osu4k_calc\ManiaMapAnalyserbyLeo_Black\。

日志：%LOCALAPPDATA%\osu4k\logs\run_*.log。

手动重跑（先关 lazer）：双击 osu4k.cmd，或 powershell-ExecutionPolicyBypass-Fileengine\run_workflow.ps1-ResetBaseline。

开关：-ReportOnly（只重出报告）、-Force（强制重建 db）、-ResetBaseline（newMaps 只统计本次真正新增）。

导入 lazer：关 lazer → CollectionManager Load 4k_collections.db → File/Save/Default osu! collection → 重开。

C 附录 C：术语表

MSD Mina Standardized Difficulty, Etterna/MinaCalc 的 4K 难度（与玩家实力同尺）。本工具取其 Overall。

ISR Interlude/YAVSRG 的难度评分（单值）。

RSR sunny rework 的难度评分（单值）。

skillset MinaCalc 的 8 个技能维度。

lnr long note ratio，长条音符占总物量的比例。

dan / 段位 人工编排的递增难度课题包（如 REFORM 1st..10th），达到通过线即视为该段位。离散、不等距、随版本漂移。

REFORM 10 = 13 现公式的单点锚定：让 REFORM 第 10 段的 scaled 约等于 13，借此固定整体刻度。

provisional 低置信度标记（数据不全的谱）。